Technical Documentation for Genesys Cloud Object Migration

Author: Bavani Selvamani

Date: 30.07.2025

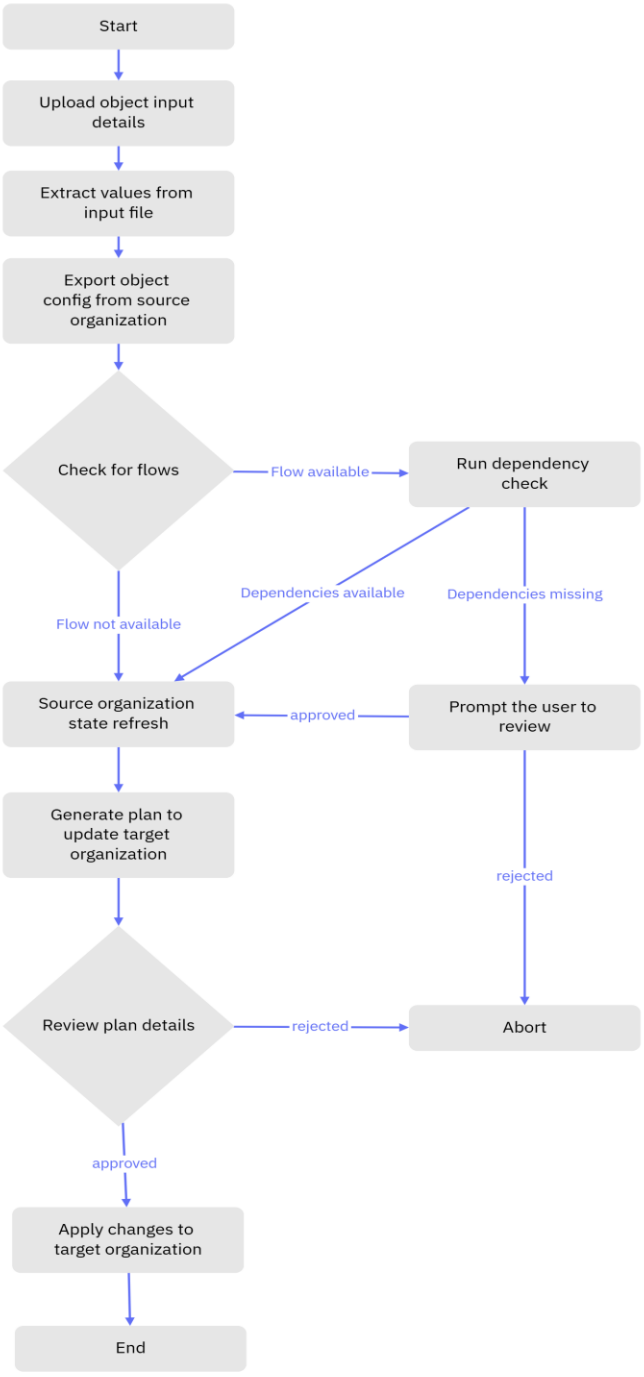
Approved By: Mohamed Shameerudheen Mohamed Haneef

Contents

4. Overview

This document describes the technical implementation of the Genesys Cloud object migration process using Terraform for export/import logic, Python for processing the exported terraform configuration , and Jenkins for automation.

5. Process Flow



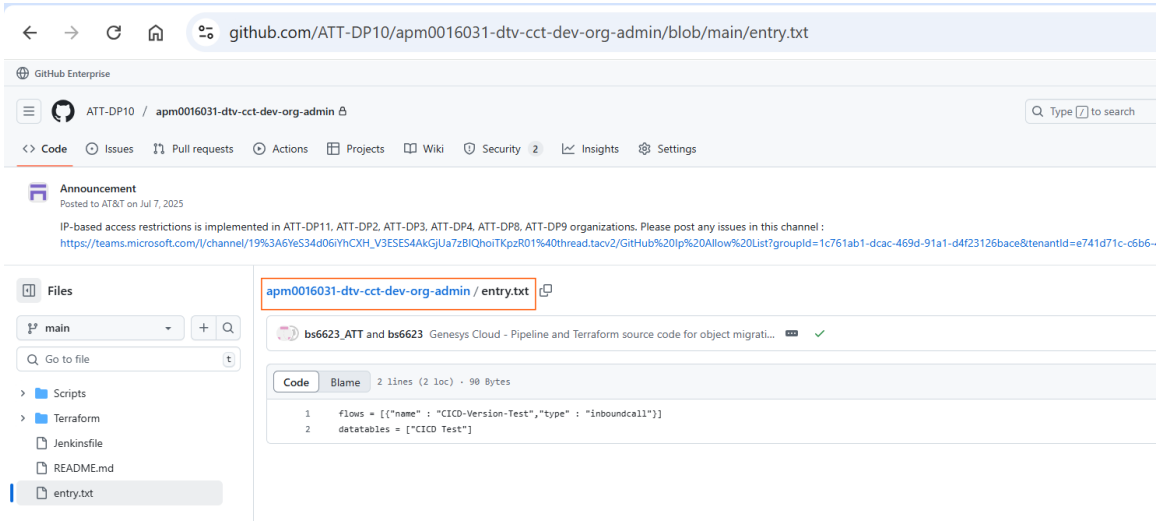
6. Technology Stack

- Terraform: v1.12.2 with Genesys Cloud Provider: 1.65.0 (Latest at the time of this document creation)
- Python: v3.6
- Genesys Cloud CLI - v133.0.0 (Latest at the time of this document creation)
- Jenkins: Declarative pipeline with credentials binding
- Git: Version control for Terraform and Python scripts

7. Deployment Instructions

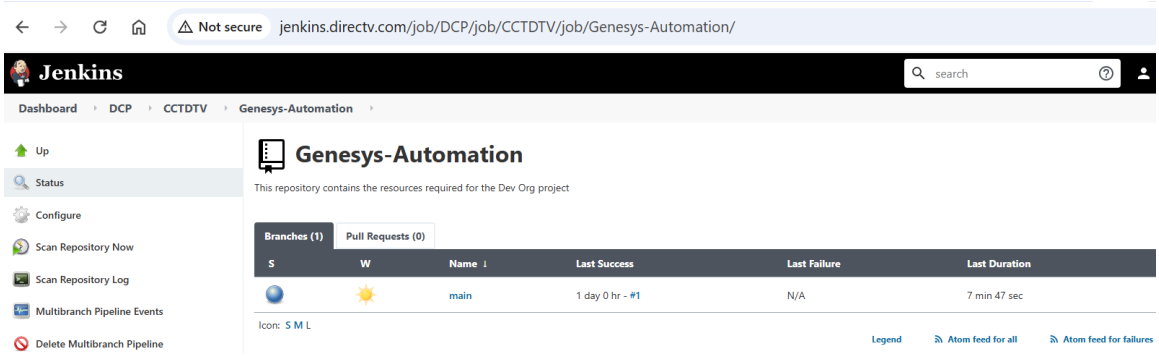
Source Code

Ensure the `entry.txt` file is correctly formatted and available in the source code GitHub repository path - https://github.com/ATT-DP10/apm0016031-dtv-cct-dev-org-admin/blob/main/entry.txt

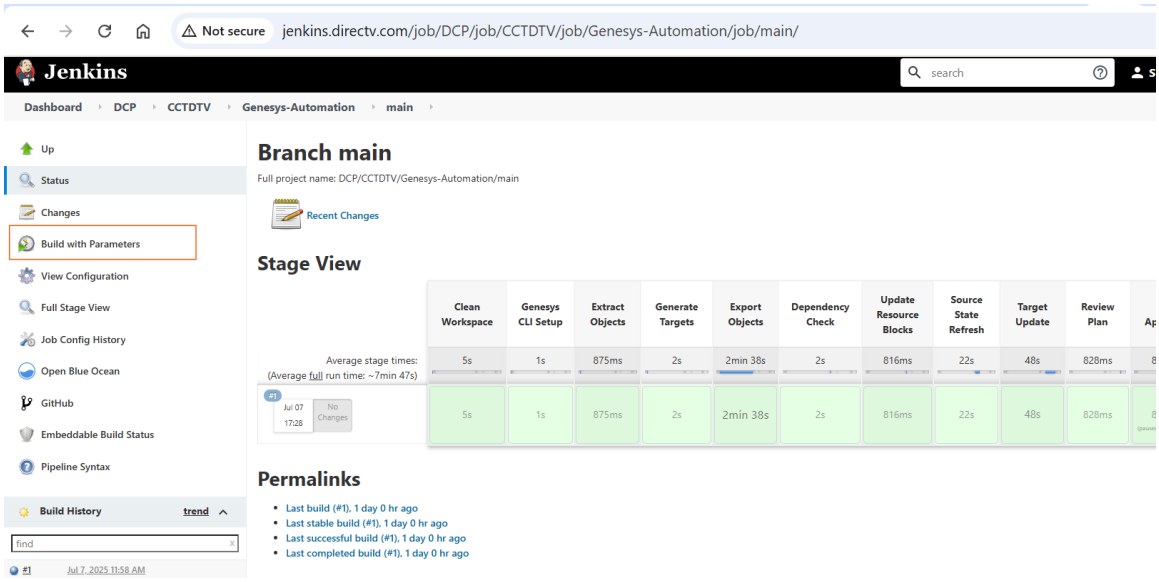


Jenkins

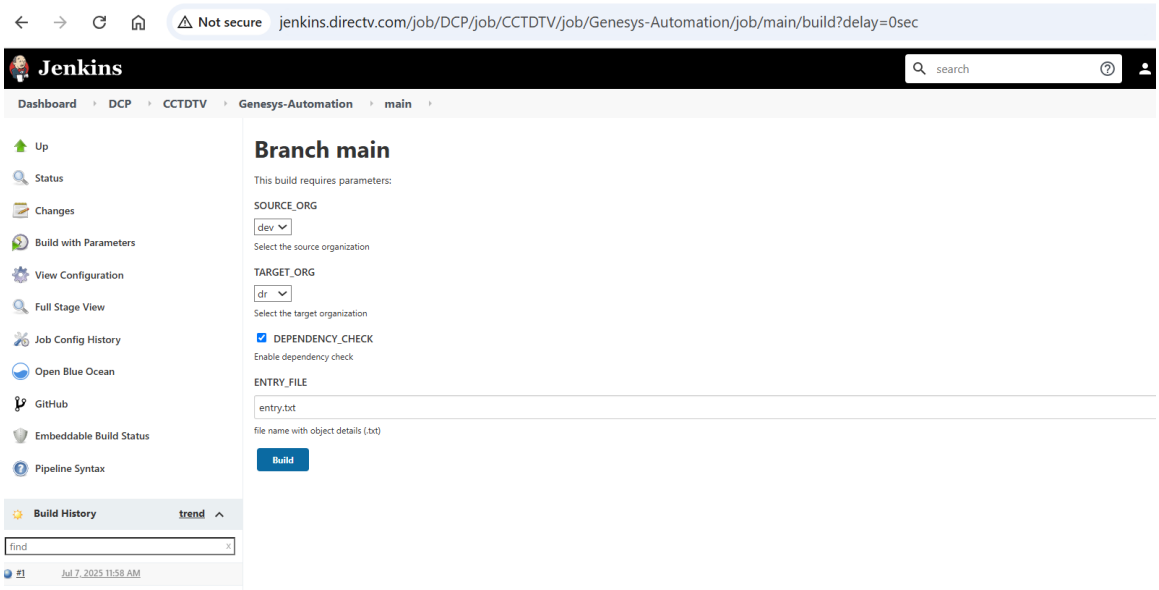
1. Navigate to the Jenkins job link: http://jenkins.directv.com/job/DCP/job/CCTDTV/job/Genesys-Automation/)



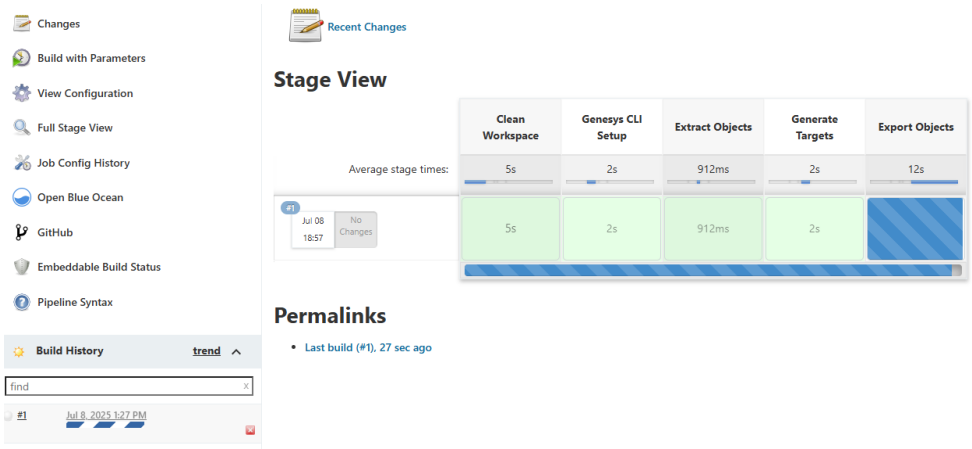
2. Select the desired branch.
3. Click ***Build with Parameters***.



4. Provide the required parameters.



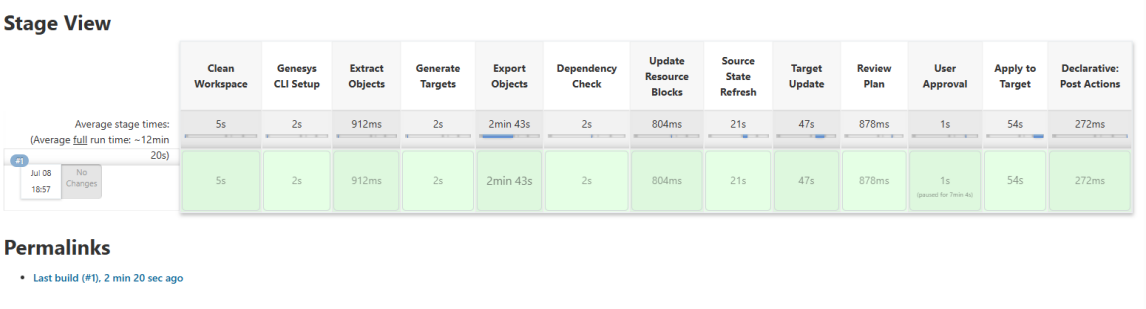
5. Click on the 'build' to start the job



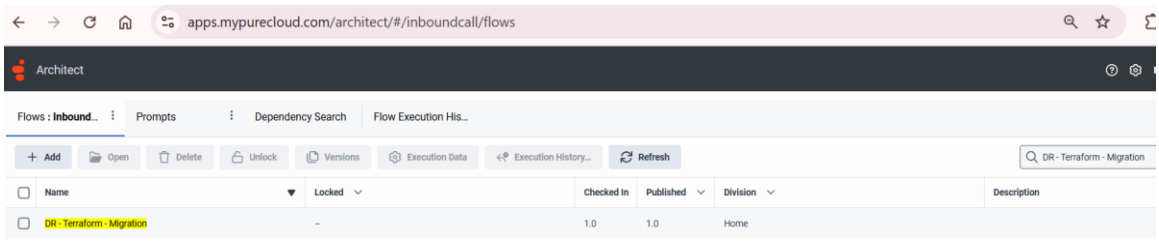
6. During the *Dependency Check* stage, if missing dependencies are found, the user is prompted to approve continuation.
7. During the *Review Plan* stage, the Terraform plan is displayed.

8. In the *User Approval* stage, the user must approve the plan to proceed with applying changes.

9. Once approved, the required changes will be completed in target organization and the pipeline will reach post actions with ‘Success’.



10. After successful deployment, the new migrated objects / changes can be verified in the target organization.



8. Implementation

5.1. Source Code

Source code repository : GitHub

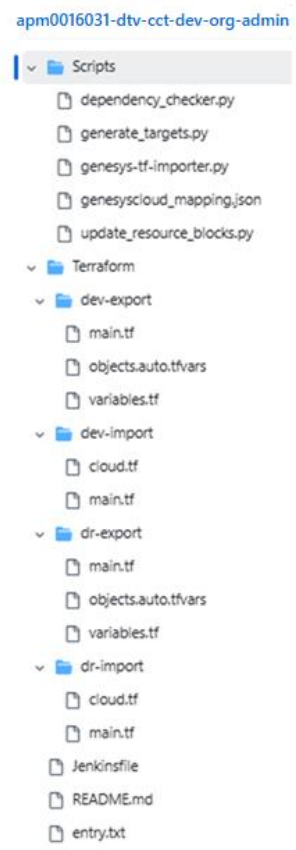
Repository Link : https://github.com/ATT-DP10/apm0016031-dtv-cct-dev-org-admin.git

Main Branch: main

This repository includes

- entry.txt - file for input data
- Jenkinsfile - Pipeline script for Jenkins
- Terraform – TF configuration files needed for export and apply on Genesys Cloud
- Scripts – Python scripts used in the pipeline set up for processing the tf files and do pre-checks.

5.1.1. Folder Structure



5.1.2. Terraform Configuration

5.1.2.1. Export object from an organization:

This section provides a detailed explanation of the Terraform configuration used to export Genesys Cloud objects from a source organization. It includes descriptions of each block and attribute defined in the main.tf, variables.tf, and objects.auto.tfvars files. The configuration leverages the Genesys Cloud Terraform provider as documented at https://registry.terraform.io/providers/MyPureCloud/genesyscloud/latest/docs (v1.65.0)

main.tf

Terraform Block

The terraform block specifies the required provider for Genesys Cloud. The 'source' attribute identifies the provider's namespace and name. The version is commented out but can be specified to lock the provider version.

Example:

```
terraform {
 required_providers {
  genesyscloud = {
   source = "MyPureCloud/genesyscloud"
  }
 }
}
```

Provider Block

The provider block configures the Genesys Cloud provider with OAuth credentials and AWS region. These values are referenced from input variables.

```
provider "genesyscloud" {
  oauthclient_id     = var.dev_client_id
  oauthclient_secret = var.dev_client_secret
  aws_region         = var.region
}
```

Attributes:

- *oauthclient_id*: OAuth client ID for authentication.
- *oauthclient_secret*: OAuth client secret for authentication.
- *aws_region*: AWS region used for Genesys Cloud deployment.

Locals Block

The locals block defines computed values used to filter which Genesys Cloud resources should be exported. It constructs lists for flows, datatables, prompts, queues, divisions, and schedule groups based on input variables. These lists are combined into *'filtered_resource*'. Additional lists define resources to replace with data sources and attributes to exclude.

```
locals {
…
}
```

Resource Block:

This resource performs the export operation. It uses the filtered resource list and configuration flags to control export behavior.

```
resource "genesyscloud_tf_export" "export" {
….
```

}

Key Attributes:

- directory: Output directory for exported files.
- include_filter_resources: List of resources to include in export.
- include_state_file: Whether to include Terraform state file (false).
- export_as_hcl: Export format (true for HCL).
- log_permission_errors: Log permission errors during export.
- use_legacy_architect_flow_exporter: Use legacy exporter (false).
- enable_dependency_resolution: Enable dependency resolution.
- replace_with_datasource: List of resources to replace with data sources.
- exclude_attributes: List of attributes to exclude from export.

variables.tf

This file defines input variables used in the Terraform configuration. These variables allow customization of the export process and support sensitive credentials and object lists.

Ex.

```
variable "variable_name" {
  default = "default_value"
}
```

Key Variables:

- dev_client_id: OAuth client ID (sensitive).
- dev_client_secret: OAuth client secret (sensitive).
- region: AWS region (default: us-west-2).
- location: Genesys Cloud location (default: usw2.pure.cloud).
- flows: List of flow objects with name and type.
- datatables: List of datatable names.
- prompts: List of prompt names.
- schedule_groups: List of schedule group names.
- queues: List of queue names.
- divisions: List of division names.

objects.auto.tfvars

This file is a placeholder for runtime input values. It's automatically loaded by Terraform and used to populate variables during execution. Users provide values for flows, datatables, prompts, etc. at runtime.

5.1.2.2. Refresh source organization state:

This section describes the Terraform configuration used to refresh the state of the source Genesys Cloud organization. It includes three key files: main.tf, cloud.tf, and genesyscloud.tf. Each file plays a specific role in setting up the provider, defining the Terraform Cloud workspace, and storing the exported object configurations.

main.tf

Provider Block

The provider block configures the Genesys Cloud provider with OAuth credentials and AWS region. These values are referenced from input variables.

Attributes:

- *oauthclient_id*: OAuth client ID for authentication.
- *oauthclient_secret*: OAuth client secret for authentication.
- *aws_region*: AWS region used for Genesys Cloud deployment.

Variables Block

This file defines input variables used in the Terraform configuration. These variables allow customization of the export process and support sensitive credentials and object lists.

Ex.

```
variable "variable_name" {
  default = "default_value"
}
```

Key Variables:

- dev_client_id: OAuth client ID (sensitive).
- dev_client_secret: OAuth client secret (sensitive).
- region: AWS region (default: us-west-2).
- location: Genesys Cloud location (default: usw2.pure.cloud).

cloud.tf – Terraform Cloud Workspace Setup

The cloud.tf file configures the Terraform Cloud backend. It specifies the organization and workspace used to manage the state remotely.

```
terraform {
  cloud {
    organization = "DTV-Terraform-Dev-Org"
```

```
  workspaces {
    name = "Dev"
  }
 }
}
```

Key components:

- organization: DTV-Terraform-Dev-Org
- workspace name: Dev

This setup ensures that Terraform operations are executed in a consistent environment with remote state management:

genesyscloud.tf – Exported Object Configuration

The genesyscloud.tf file is generated as a result of the Terraform export operation. It contains HCL configuration blocks for Genesys Cloud resources such as flows, queues, prompts, and datatables.

Key components:

- Required_providers block with source and version
- Multiple resource blocks representing exported Genesys Cloud objects

This file serves as the foundation for importing and refreshing the source organization state using Terraform.

5.1.2.3. Update to target organization:

This section describes the Terraform configuration used to update the target Genesys Cloud organization. It includes variable declarations, provider setup, Terraform Cloud workspace configuration, and the resultant HCL configuration generated from the source export process.

main.tf

Provider Block

The provider block configures the Genesys Cloud provider with OAuth credentials and AWS region. These values are referenced from input variables.

Attributes:

- *oauthclient_id*: OAuth client ID for authentication.
- *oauthclient_secret*: OAuth client secret for authentication.

- *aws_region*: AWS region used for Genesys Cloud deployment.

Variables Block

This file defines input variables used in the Terraform configuration. These variables allow customization of the export process and support sensitive credentials and object lists.

Ex.

```
variable "variable_name" {
  default = "default_value"
}
```

Variables:

- dr_client_id: OAuth client ID for the target organization (sensitive).
- dr_client_secret: OAuth client secret for the target organization (sensitive).
- region: AWS region used for Genesys Cloud provider configuration (default: us-east-1).
- location: Genesys Cloud location domain (default: usw2.pure.cloud).
- exclude_attributes: List of resource attributes to exclude during import or apply operations.

cloud.tf – Terraform Cloud Workspace Configuration

The cloud.tf file configures the Terraform Cloud backend. It specifies the organization and workspace used for managing the state remotely.

```
terraform {
  cloud {
    organization = "DTV-Terraform-Dev-Org"
    workspaces {
      name = "DR"
    }
  }
}
```

This configuration ensures that Terraform operations are executed in the 'DR' workspace under the specified organization.

genesyscloud.tf – Exported Object Configuration

The genesyscloud.tf file is generated as a result of the Terraform export operation. It contains HCL configuration blocks for Genesys Cloud resources such as flows, queues, prompts, and datatables.

Key components:

- Required_providers block with source and version
- Multiple resource blocks representing exported Genesys Cloud objects

This file serves as the foundation for importing and refreshing the source organization state using Terraform.

5.1.3. Python Scripts

5.1.3.1. generate_targets.py

Purpose

The script generate_targets.py is designed to dynamically generate Terraform target flags for specific Genesys Cloud resources. It uses the Genesys Cloud CLI to query resource metadata and constructs -target flags for Terraform operations such as refresh, plan, and apply.

Input

The script accepts resource lists via environment variables and a Genesys Cloud CLI profile name via command-line argument:

- flows: JSON list of flow objects with 'name' and 'type'.
- datatables: JSON list of datatable names.
- prompts: JSON list of prompt names.
- schedule_groups: JSON list of schedule group names.
- queues: JSON list of queue names.
- divisions: JSON list of division names.
- profile: CLI profile name passed as sys.argv[1].

Logic Overview

1. Load environment variables and parse them as JSON.
2. For each datatable name:
   - Query its ID using the CLI.
   - Generate a target for the datatable and its rows.
3. For each flow:
   - Construct a target using its type and name.
4. For each prompt, schedule group, queue, and division:
   - Construct a Terraform target using the resource type and sanitized name.
5. Combine all targets into a single space-separated string and print it.

Genesys Cloud CLI commands used:

1. List the datatables with the given name
   > gc flows datatables list --name "<name>" --profile <profile> --autopaginate
2. List the datatable rows for the given datatable id
   > gc flows datatables rows list "<datatable_id>" --profile <profile> --autopaginate

Output

The script prints a space-separated string of Terraform -target flags. These flags are used in CI/CD pipelines to selectively refresh or apply specific resources.

Error Handling

If the CLI command fails or returns invalid JSON, the script logs a message and skips that resource.

5.1.3.2. dependency_checker.py

Purpose

The script 'dependency_checker.py' is designed to identify and validate dependencies for Genesys Cloud flows. It ensures that all referenced resources in a flow are present in the Terraform configuration before deployment. This helps prevent runtime errors due to missing dependencies.

Logic Overview

1 Load a predefined mapping of Genesys Cloud object types from 'genesyscloud_mapping.json'.
2 Accept a JSON string of flow definitions as input.
3 For each flow, retrieve its ID and published version using the Genesys Cloud CLI.
4 Fetch consumed resources (dependencies) for the flow.
5 Filter dependencies to include only relevant types.
6 Save all dependencies to 'found.json'.
7 Extract dependencies and compare them against Terraform resource definitions in 'genesyscloud.tf'.
8 Identify and report missing resources.

Input

The script expects a single command-line argument: a JSON string representing a list of flows with 'name' and 'type'.
Example:
python dependency_checker.py '[{"name": "FlowA", "type": "INBOUNDCALL"}]'

Genesys Cloud CLI commands used:

1. List flows:
   > gc flows list --name <flow_name> --profile dev --varType <type> --filtercondition name==<flow_name>
2. List consumed resources:
   > gc architect dependencytracking consumedresources list --id <flow_id> --version <version> --objectType <type>FLOW --profile dev

Output

1. 'found.json': Contains all relevant dependencies for each flow.
2. 'missing_dependencies_by_flow.json': Lists dependencies that are not found in the Terraform configuration.
3. Console output: Displays missing dependencies and status messages.

Error Handling

The script includes error handling for subprocess failures and JSON parsing errors. If a CLI command fails or returns invalid JSON, the script logs the error and continues processing other flows.

Integration with Terraform

The script reads the Terraform configuration file 'genesyscloud.tf' to extract defined resources. It compares these against the dependencies found in Genesys Cloud to ensure all required resources are declared. This validation step is crucial before applying Terraform changes.

5.1.3.3. update_resource_blocks.py

Purpose

The script 'update_resource_blocks.py' is designed to modify Terraform resource blocks for Genesys Cloud routing queues. Specifically, it appends a lifecycle block to each 'genesyscloud_routing_queue' resource to ignore changes to certain attributes that are managed outside of Terraform or are expected to have different configurations in each organization.

Logic Overview

1. The script defines a function `append_lifecycle_block_to_queues` that uses regular expressions to locate all routing queue resource blocks.
2. For each matched block, it checks if a lifecycle block is already present.

3. If not, it appends a lifecycle block that ignores changes to 'members' and 'last_agent_routing_mode'.
4. The script reads the Terraform file located at './Terraform/dev-export-result/genesyscloud.tf'.
5. It applies the transformation and writes the updated content back to the same file.

File Handling

The script opens the target Terraform file in read mode to load its content. After processing, it writes the updated content back to the same file in write mode. This ensures that the lifecycle blocks are added directly to the existing configuration.

Error Handling

The script includes basic error handling for file operations. If the specified file is not found, it catches FileNotFoundError and prints an error message indicating the missing file path.

5.1.3.4. genesys-tf-importer.py

Purpose

The genesys-tf-importer.py script automates the generation of Terraform import blocks for existing Genesys Cloud resources under Terraform management. It scans Terraform configuration files, identifies unmanaged resources, and uses the Genesys Cloud CLI to fetch resource IDs for import.

Logic Overview

1. Accepts the target environment (e.g., 'dev' or 'dr') as a command-line argument.
2. Clears or initializes two output files: import.tf and import.sh.
3. Walks through all .tf files in the current directory and subdirectories.
4. Identifies resource blocks and checks if they are already managed in Terraform state.
5. Extracts the resource name from the block for lookup.
6. Uses CLI commands mapped per resource type to fetch the resource ID.
7. Writes Terraform import blocks to import.tf and shell commands to import.sh.

CLI Integration

The script uses the Genesys Cloud CLI (gc) to query resources by name and retrieve their unique IDs. Each Terraform resource type is mapped to a corresponding CLI command. For example:

- genesyscloud_routing_queue → gc routing queues
- genesyscloud_flow → gc flows

- genesyscloud_architect_user_prompt → gc architect prompts
  Special handling is implemented for datatable rows which require composite IDs.

File Generation

The script generates two files:

- import.tf: Contains Terraform import blocks for each unmanaged resource.
- import.sh: Contains shell commands to execute terraform import for each resource. (In case the CICD Pipeline needs to execute the import commands alone.)

Error Handling

The script includes error handling for subprocess failures and JSON parsing errors. It prints informative messages when CLI commands fail or when resource blocks cannot be matched or parsed.

Integration with Terraform

This script supports Terraform workflows by enabling the import of existing Genesys Cloud resources into state. It ensures that resources defined in configuration files are properly tracked and managed by Terraform.

5.1.4. Jenkins Pipeline

This section provides a detailed explanation of the Jenkins pipeline used for automating the migration of Genesys Cloud objects between organizations. The pipeline integrates Terraform and Python scripts to extract, transform, validate, and apply configuration changes across environments. It follows CI/CD best practices including parameterization, environment isolation, dependency checks, and user approval gates.

5.1.4.1. Pipeline Parameters

The pipeline accepts the following parameters to control execution behavior:

- SOURCE_ORG: Specifies the source Genesys Cloud organization, defaults to ‘dev’
- TARGET_ORG: Specifies the target Genesys Cloud organization, defaults to ‘dr’
- DEPENDENCY_CHECK: Boolean flag to enable or disable dependency validation before applying changes, defaults to ‘true’
- ENTRY_FILE: Specifies the file name which contains the object details to be processed, defaults to ‘entry.txt’

5.1.4.2. Environment Variables

Environment variables are configured to securely pass credentials and configuration values to Terraform and Python scripts:

- TF_VAR_dev_client_id & TF_VAR_dev_client_secret: OAuth credentials for the source organization.
- TF_VAR_dr_client_id & TF_VAR_dr_client_secret: OAuth credentials for the target organization.
- TF_VAR_dev_location & TF_VAR_dr_location: API endpoints for each organization.
- TF_TOKEN_app_terraform_io: Token for Terraform Cloud integration.
- GC_CLI_PATH / PATH: Path configuration for Genesys CLI.
- DEPENDENCY_CHECK: Propagated from pipeline parameters.

5.1.4.3. Pipeline Stages

Stage: Clean Workspace

Deletes previous workspace files and checks out the latest source code.

Stage: Genesys CLI Setup

Downloads and configures the Genesys CLI tool for API interactions.

Stage: Extract Objects

Reads input variables from a file and generates Terraform variable definitions.

Stage: Generate Targets

Runs a Python script to determine which Terraform resources should be targeted.

Stage: Export Objects

Initializes and applies Terraform configuration to export Genesys Cloud objects.

Stage: Dependency Check

Validates that all required dependencies for flows are present using a Python script.

Stage: Update Resource Blocks

Updates Terraform resource blocks based on the exported data.

Stage: Source State Refresh

Refreshes the source environment with the latest exported configuration.

Stage: Target Update

Prepares the target environment by copying and transforming exported resources.

Stage: Review Plan

Displays the Terraform plan output for review.

Stage: User Approval

Prompts the user to approve the Terraform plan before applying changes.

Stage: Apply to Target

Applies the approved Terraform changes to the target organization.

5.1.4.4. Post Actions

The pipeline includes post-execution actions to log the outcome:

- On failure: Logs 'Pipeline failed!'.

- On success: Logs 'Pipeline completed'.

5.1.4.5. CI/CD Best Practices

This pipeline follows CI/CD best practices including:

- Secure credential management using Jenkins credentials binding.
- Parameterized execution for flexibility across environments.
- Automated validation and user approval gates to prevent misconfigurations.
- Modular use of Python scripts and terraform for maintainability.

9. Security Considerations

- Client OAuth Credentials for Genesys Cloud Organizations are stored in Jenkins credentials (needs Admin privileges to read/write)
- GC CLI Profile configuration for Genesys Cloud Organizations is stored in Jenkins config files (needs Admin privileges to read/write)
- Terraform configuration files, Python scripts are managed in GitHub Repository, accessible only using AT&T Sign on.
- Terraform State is managed in Terraform HCP Cloud, each Genesys Cloud Organization state isolated in separate workspaces.

10. Conclusion

The Genesys Cloud Object Migration framework outlined in this document provides a robust, scalable, and secure approach to managing configuration changes across cloud environments. By leveraging Terraform for infrastructure as code, Python for dynamic scripting, and Jenkins for automation, the solution ensures consistency, traceability, and operational efficiency.

This documentation serves as a reference for development and operations teams to implement, maintain, and evolve the migration pipeline. Future enhancements may include support for additional Genesys Cloud objects, improved error handling, and integration with monitoring tools.

END.